NLP Adaptive Tutor

Author: Joel Assessment: Project (Component 1, 100%) Date: 2025-12-16

Assessment and Submission Details

- Deadline: 12:00 (noon) on 16/ 12/ 2025.
- Submission: Moodle submission link.
- Required format: ZIP containing all source code, supporting documentation, figures, and dataset links. Any code missing from the ZIP is not marked.
- Generative AI: permitted for support with understanding and small assistance, but must be declared with tool name, version, and free/ paid status.

Abstract

This project delivers a compact NLP-based tutoring system for short learner responses. It combines language identification, syntax checks, semantic similarity, named entity recognition (NER), and fluency scoring into a single pipeline. A FastAPI backend exposes the analysis, and a lightweight web UI presents feedback to the learner.

Project Idea and Objectives

I chose to build an NLP tutor for short sentence practice, aligned with dialogue systems and human-computer interaction. The objectives were to:

- Detect whether the learner wrote in the expected language (EN/ ES/ PL).
- Provide basic syntax feedback when the language matches.
- Score meaning similarity against a target sentence.
- Show nearest example targets as hints, including a cross-language view.
- Extract named entities and noun phrases to support feedback.
- Provide a fluency signal using a simple n-gram language model.

What Was Implemented

- End-to-end pipeline in `nlp_ tutor/ pipeline.py` combining language ID, syntax, semantics, NER, and fluency.
- Language detection using both TF-IDF + logistic regression and a character BiLSTM classifier.
- Semantic similarity using TF-IDF cosine and a transformer embedding model.
- Cross-language nearest targets, returning one EN, one PL, and one ES example when available.
- spaCy-based syntax checks and NER extraction.
- Fluency estimation using an n-gram language model and perplexity bands.
- FastAPI endpoints for tutor evaluation and semantic scoring.
- Solid.js UI showing scores, nearest targets, syntax, fluency, and NER panels.

Data and Resources

- Custom lesson bank (12 items) in `nlp_ tutor/ resources/ lesson_ bank.csv` with columns: lesson_ id, item_ id, topic, prompt_ en, target_ es, target_ en, target_ pl, gloss_ en.
- WiLI 2018 language identification dataset (EN/ ES/ PL) in `data/ raw/ wili-2018` (used to train language ID models).
- NLTK corpora for fluency: Brown (EN) and CESS-ESP (ES).
- Optional sentiment baseline data: IMDB dataset from Kaggle (not central to the tutor pipeline).
- Pretrained models: spaCy language models (en_ core_ web_ sm, es_ core_ news_ sm, pl_ core_ news_ sm) and Sentence-Transformers `paraphrase-multilingual-MiniLM-L12-v2`.

Methods

Preprocessing

- Lowercasing and whitespace normalization.
- NLTK tokenization, with optional lemmatization for English.

Language Detection

- Baseline: character TF-IDF with logistic regression.

- Neural: character BiLSTM classifier for robustness on short texts.
- Top-k outputs for confidence and an uncertainty gate in the policy.

Semantics and Retrieval

- TF-IDF cosine similarity and transformer embedding cosine similarity.
- Nearest target retrieval by backend and an additional cross-language list.

Syntax and NER

- Syntax heuristics and checks using spaCy pipelines.
- NER extraction (entities and noun phrases) from spaCy.
- When detected language is confident but does not match expected, NER uses the detected language.

Fluency

- N-gram language models with perplexity-based bands.
- Calibration on in-domain lesson texts when possible.

Dialogue Policy

- Rule-based policy gates on language, syntax, meaning, then fluency to produce a clear next action.

System Architecture

- Core pipeline: `nlp_ tutor/ pipeline.py` orchestrates all NLP signals.
- Semantic engine: `nlp_ tutor/ lessons.py` fits per-language scorers over lesson targets.
- API: FastAPI in `api/ ` exposes `/ tutor/ evaluate` and `/ semantic/ score`.
- UI: Solid.js frontend in `web/ ` renders feedback panels.

Evaluation and Results

These results come from local runs documented in the project README. They are small-scale sanity checks rather than large benchmark evaluations.

Language Detection (WiLI 2018 test sample)

- Setup: 500 samples per class (EN, ES, PL), total N=1500.
- Baseline (char TF-IDF + logreg): Accuracy 0.9927, Macro F1 0.9927.
- Char BiLSTM: Accuracy 0.9440, Macro F1 0.9437.

Semantic Similarity (lesson bank sanity check)

- Setup: exact target vs random negative target (12 items).
- TF-IDF: mean positive 1.0000, mean negative 0.0133, pairwise accuracy 1.00.
- Transformer: mean positive 1.0000, mean negative 0.2634, pairwise accuracy 1.00.

Learning Reflection

The project aligns with the weekly topics and activities in the module:

- 23-09-2025 Introduction to NLP: defined task scope and the end-to-end pipeline.
- 30-09-2025 Text Preprocessing: applied normalization and tokenization techniques.
- 07-10-2025 Language Models and N-grams: implemented the fluency scorer.
- 14-10-2025 Syntax and Parsing: added syntax checks with spaCy.
- 21-10-2025 Semantics and Word Representations: used TF-IDF and transformer embeddings.
- 04-11-2025 Sequence Models: implemented the character BiLSTM language detector.
- 11-11-2025 Transformers and Attention: applied transformer embeddings for similarity.
- 18-11-2025 Information Extraction: implemented NER and noun phrase extraction.
- 25-11-2025 Text Classification and Sentiment Analysis: optional sentiment baseline.
- 02-12-2025 Dialogue Systems and Chatbots: structured feedback and policy gating.

Research and Reading

- Module lecture slides, labs, and the reading list (research papers and book chapters) provided via Moodle.
- Documentation and tutorials for spaCy, NLTK, scikit-learn, PyTorch, and Sentence-Transformers.
- Dataset documentation for WiLI 2018 and the IMDB Kaggle dataset.

Limitations and Incomplete Work

- The lesson bank is small and domain-specific, limiting semantic variety.
- The semantic evaluation is a sanity check and does not test paraphrase robustness.
- Polish fluency is not fully supported without an external PL corpus.
- NER depends on spaCy models; performance varies across languages and domains.
- No user study or large-scale evaluation with learner data was performed.

Future Work

- Expand the lesson bank with paraphrases and learner error examples.
- Add stronger evaluation on real learner data and calibrate thresholds.
- Train or adapt NER models to domain-specific entities.
- Add multilingual fluency corpora for Polish and other languages.

AI Usage Statement

I used a generative AI tool for minor support (rewriting sections for clarity, checking structure, and improving wording). AI Tool: OpenAI Codex CLI (GPT-5) Access: Paid Specific uses: Editing and restructuring report text; checking for consistency.

References (Newman-Harvard Style)

- Thoma, M. (2018) WiLI-2018: Wikipedia Language Identification dataset. Available at: https:/ / zenodo.org/ records/ 840106 (Accessed: 2025-12-16).
- Maas, A.L. et al. (2011) Learning word vectors for sentiment analysis. Available at: https:/ / www.kaggle.com/ datasets/ lakshmi25npathi/ imdb-dataset-of-50k-movie-reviews (Accessed: 2025-12-16).
- Reimers, N. and Gurevych, I. (2019) Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Available at: https:/ / arxiv.org/ abs/ 1908.10084 (Accessed: 2025-12-16).
- Wolf, T. et al. (2020) Transformers: State-of-the-art natural language processing. Available at: https:/ / huggingface.co/ transformers/ (Accessed: 2025-12-16).
- spaCy (2024) Industrial-strength Natural Language Processing in Python. Available at: https:/ / spacy.io/ (Accessed: 2025-12-16).
- NLTK Project (2024) Natural Language Toolkit. Available at: https:/ / www.nltk.org/ (Accessed: 2025-12-16).
- Pedregosa, F. et al. (2011) Scikit-learn: Machine Learning in Python. Available at: https:/ / scikit-learn.org/ (Accessed: 2025-12-16).
- Paszke, A. et al. (2019) PyTorch: An imperative style, high-performance deep learning library. Available at: https:/ / pytorch.org/ (Accessed: 2025-12-16).
- Module lecture materials (2025) NLP module slides, lab notes, and curated web links provided via Moodle (Accessed: 2025-12-16).